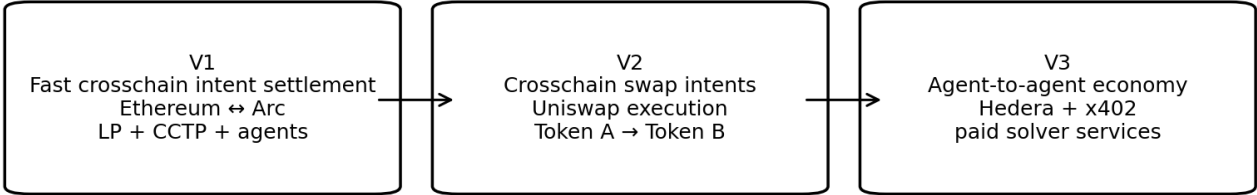


# ARCAIDIA

## Agentic Crosschain Intent & Liquidity Network

### ETHOnline 2026 - Build Specification

V1 is the submission-critical scope and is bidirectional from day one: Ethereum <-> Arc. V2 (Uniswap) begins only after V1 works end-to-end. V3 (Hedera/x402) is a stretch/post-hackathon extension.



*Prepared as a direct handoff to Claude Code*

# 1. Executive decision

Arcaidia V1 is not a new canonical bridge. It is a fast settlement layer above Circle CCTP. The user's source USDC is committed to CCTP first. Only after that commitment is observed and independently verified may an autonomous liquidity agent advance destination USDC from an destination-chain LP vault. CCTP then completes asynchronously and replenishes the LP.

Core invariant: Fast settlement accelerates the user experience; canonical settlement remains the source of economic finality.

- Onchain contracts hold funds, enforce replay protection and define authoritative state.
- Offchain agents observe, price risk and decide whether to advance liquidity.
- The Graph is the live discovery/state layer, but never the sole authority for moving pooled funds.
- Circle Agent Wallets provide bounded machine signing authority.
- CCTP is initiated before the fast fill, materially reducing LP settlement risk.
- If no solver fast-fills, canonical CCTP delivery remains the fallback path.

# 2. Scope and versions

| Version | Purpose                                                                                                                        | Hackathon rule                             |
|---------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| V1      | Ethereum ↔ Arc USDC fast intent settlement using LP liquidity, The Graph, autonomous agents, Circle Agent Wallets and CCTP     | Must ship and stand alone                  |
| V2      | Generalised crosschain swap intents using Uniswap: user specifies desired destination asset; solver determines execution route | Only after V1 passes end-to-end acceptance |
| V3      | Hedera/x402 machine payments for paid solver/discovery services                                                                | Stretch or post-hackathon                  |

V1 contracts should use a general intent schema where practical, but execution must be allowlisted to USDC and Ethereum/Arc so V2 can be added without destabilising V1.

# 3. V1 actors and trust boundaries

| Actor                  | Location                                 | Responsibility / trust                                                                                    |
|------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| User + Privy wallet    | Offchain signer / onchain address        | Creates intent and authorises USDC spend.                                                                 |
| ArcaidiaIntentRouter   | Ethereum onchain                         | Pulls USDC, creates immutable intent, initiates CCTP, emits intent event.                                 |
| The Graph              | Offchain indexing/data                   | Discovers live intents and exposes liquidity/settlement state to agents. Not an authorisation oracle.     |
| Liquidity Agent        | Offchain                                 | Independently verifies source transaction, prices fee/risk, accepts or rejects, signs fill authorisation. |
| Circle Agent Wallet    | Offchain key authority / onchain address | Signs bounded agent actions under wallet policies.                                                        |
| ArcaidiaLiquidityVault | Arc onchain                              | Holds LP USDC; verifies authorised fill, replay protection, limits and transfers fast USDC.               |
| Settlement Agent       | Offchain                                 | Monitors CCTP and completes                                                                               |

|                    |             |                                                                                  |
|--------------------|-------------|----------------------------------------------------------------------------------|
|                    |             | destination receive/reconciliation.                                              |
| SettlementReceiver | Arc onchain | Receives canonical USDC and routes it to LP reimbursement or recipient fallback. |

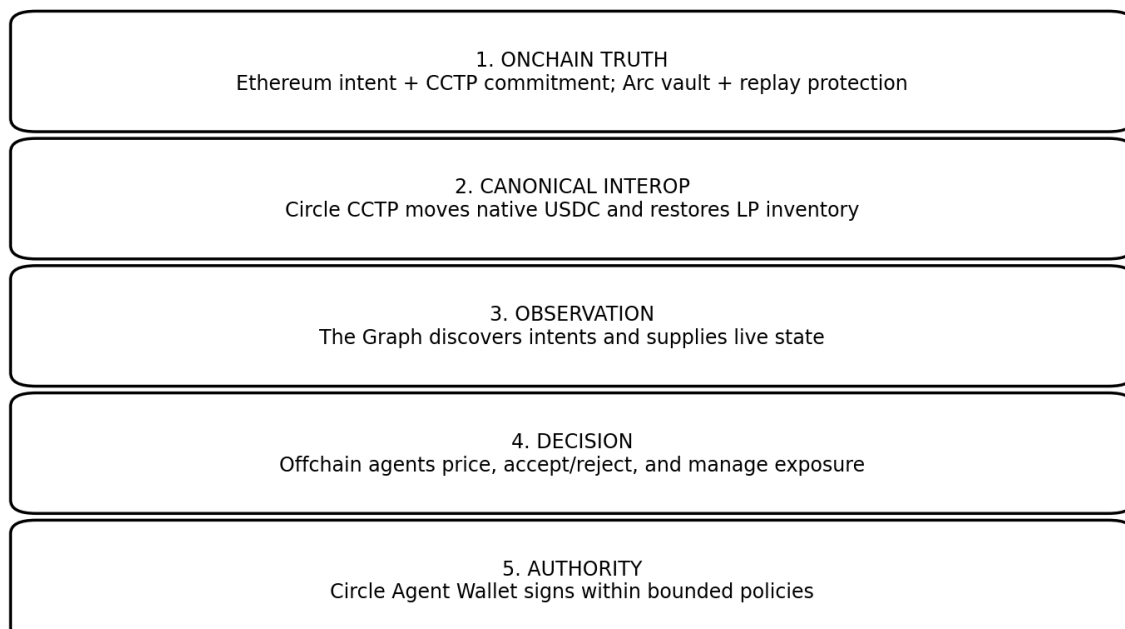


Figure 1. V1 trust hierarchy.

## 4. V1 architecture

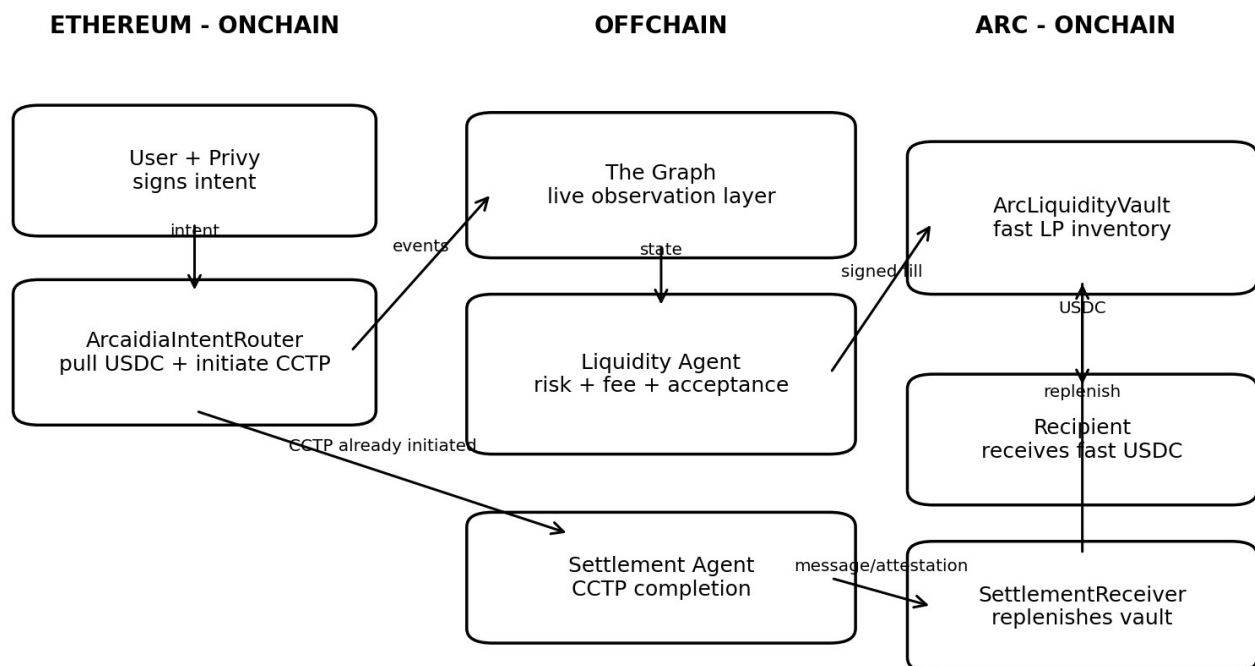
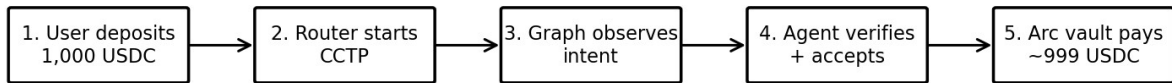


Figure 2. Onchain/offchain boundaries and the two settlement paths.

Terminology: do not call Arcadia agents 'validators'. Use Liquidity Agent, Solver Agent and Settlement Agent. They do not participate in chain consensus.

## 5. Exact V1 transaction sequence

### FAST USER PATH



**USER EXPERIENCE COMPLETE**

### CANONICAL / LP PATH

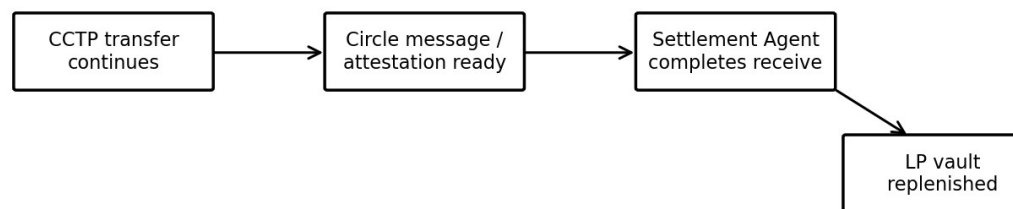


Figure 3. Fast user settlement and asynchronous canonical settlement.

1. Authenticate. User signs in with Privy and obtains/uses an Ethereum wallet.
2. Construct intent. Frontend defines input token, amount, source chain, destination chain, recipient, max fee, deadline and nonce.
3. Commit source funds. The Ethereum ArcadiaIntentRouter pulls USDC and, in the same source transaction where feasible, initiates the CCTP Ethereum→Arc transfer. It emits IntentCreated with intentId and CCTP reference/message metadata.
4. Observe. The Graph indexes the real IntentCreated event and relevant vault/settlement state. The Liquidity Agent consumes this live feed.
5. Verify. Before risking LP capital, the Liquidity Agent independently checks Ethereum RPC receipts/logs: successful router call, immutable parameters, USDC amount, recipient, Arc destination, CCTP initiation, confirmations and no prior fill.
6. Decide. The agent evaluates Arc available liquidity, reserve floor, outstanding unsettled exposure, intent size, user fee ceiling, settlement backlog and risk limits.
7. Authorise. If accepted, the Circle Agent Wallet signs an EIP-712 FillAuthorization containing intentId, source chain, source transaction hash, recipient, input/output amount, fee, expiry and nonce.
8. Fast fill. ArcadiaLiquidityVault verifies signer authorisation, replay protection, expiry, limits and liquidity, marks intent consumed, then transfers output USDC to recipient.
9. User completion. UI marks FAST\_FILLED only after Arc transaction confirmation. The user is done.
10. Canonical completion. CCTP continues independently. Settlement Agent submits/observes the destination message flow. SettlementReceiver receives canonical USDC.
11. Reimburse or fallback. If the intent was fast-filled, canonical USDC replenishes LP inventory. If it was not fast-filled, canonical USDC is delivered to the intended recipient.
12. Finalise. Intent becomes SETTLED after canonical receive and reconciliation. The Graph indexes the final state.

## 6. Source-side contract: ArcaidiaIntentRouter

The source router must make an intent eligible for fast fill only after source funds are irrevocably committed to the canonical settlement path under the configured confirmation policy.

Minimum intent data:

| Field                                 | Purpose                                                                           |
|---------------------------------------|-----------------------------------------------------------------------------------|
| <b>intentId</b>                       | Deterministic unique identifier; replay key across chains.                        |
| <b>sender / recipient</b>             | Source owner and destination beneficiary.                                         |
| <b>inputToken / amount</b>            | V1 allowlisted to native USDC only.                                               |
| <b>sourceChain / destinationChain</b> | V1 allowlisted Ethereum→Arc; reverse direction optional after one-way path works. |
| <b>maxFeeBps</b>                      | Hard user ceiling.                                                                |
| <b>deadline</b>                       | No new fast fill after expiry.                                                    |
| <b>nonce</b>                          | User replay protection.                                                           |
| <b>cctpRef</b>                        | Message hash/nonce/reference needed to correlate canonical settlement.            |

- Intent economic fields are immutable after creation.
- Router emits IntentCreated only after successful USDC pull and CCTP initiation.
- No ordinary 'cancel and withdraw source USDC' path after CCTP commitment.
- Source event must contain enough data for Graph indexing and independent RPC verification.
- V1 should cap intent size and total in-flight value.

## 7. Agent verification and decision model

The Graph is load-bearing for discovery and live state, but the agent must independently verify the source transaction before authorising LP funds. This prevents a compromised indexer or stale query from being sufficient to drain the vault.

Mandatory source checks:

- Transaction succeeded and targeted the approved ArcaidiaIntentRouter.
- IntentCreated exists and fields match the candidate intent.
- CCTP transfer was initiated for the same amount/destination flow.
- Token is approved USDC and destination is Arc.
- Source transaction meets configured confirmation threshold.
- Intent ID / source transaction has not already been filled.
- Deadline is still valid.

Deterministic risk inputs:

| Signal                                       | Example policy                                                              |
|----------------------------------------------|-----------------------------------------------------------------------------|
| <b>Available destination-chain liquidity</b> | Must cover output while preserving reserve floor.                           |
| <b>Single-intent cap</b>                     | Reject above configured maximum.                                            |
| <b>Outstanding unsettled exposure</b>        | Reject if post-fill exposure exceeds cap.                                   |
| <b>Vault utilisation</b>                     | Higher utilisation increases fee or causes rejection.                       |
| <b>CCTP backlog / oldest unsettled age</b>   | Increase fee or pause new fills when settlement is delayed.                 |
| <b>User maxFeeBps</b>                        | Quoted fee must never exceed it.                                            |
| <b>Confirmation policy</b>                   | May scale with intent size; hackathon demo may use a low testnet threshold. |

The LLM, if used, may explain or recommend parameters. Capital-safety constraints and final ACCEPT/REJECT gates must remain deterministic and unit tested.

## 8. FillAuthorization and destination LiquidityVault

The agent should not have an unrestricted 'send vault money' method. It signs a narrow EIP-712 authorisation which the destination contract verifies.

Recommended signed payload:

| Field                               | Rule                                            |
|-------------------------------------|-------------------------------------------------|
| <b>intentId</b>                     | Must be unused.                                 |
| <b>sourceChainId / sourceTxHash</b> | Binds fill to verified source commitment.       |
| <b>recipient</b>                    | Cannot be changed by submitter.                 |
| <b>inputAmount / outputAmount</b>   | Output reflects quoted fee; hard caps enforced. |
| <b>feeAmount</b>                    | Must satisfy protocol and user constraints.     |
| <b>expiry</b>                       | Short-lived, e.g. 30-60 seconds.                |
| <b>nonce</b>                        | Agent replay protection.                        |

ArcadiaLiquidityVault checks the authorised solver signer, intentId unused, nonce unused, expiry, amount caps, fee limits, available liquidity and pause state. It marks the intent consumed before transferring USDC.

Circle Agent Wallet policy should additionally restrict allowed contract(s), asset(s), per-transaction value and daily value. This creates two independent control layers: wallet policy plus smart-contract policy.

## 9. Dual settlement state and fallback

Never represent completion with one boolean. User settlement and canonical settlement are separate facts.

| Fast state         | Canonical state | Meaning                                                               |
|--------------------|-----------------|-----------------------------------------------------------------------|
| <b>PENDING</b>     | PENDING         | Intent committed; no destination advance yet.                         |
| <b>FAST_FILLED</b> | PENDING         | Recipient has LP USDC; LP carries receivable against CCTP.            |
| <b>PENDING</b>     | SETTLED         | No fast fill; canonical funds should be routed to recipient fallback. |
| <b>FAST_FILLED</b> | SETTLED         | Recipient was paid early and LP has been replenished.                 |

Fallback invariant: if no agent fast-fills, the canonical CCTP leg must still deliver the user's funds on Arc.

Arcadia fast settlement is therefore an acceleration layer, not a dependency for successful canonical transfer.

## 10. Liquidity provisioning and economics

For the one-way V1 demo, only Arc requires fast-fill LP inventory. Ethereum requires the source router/CCTP path, not a fast-liquidity vault. Add an Ethereum LP vault only when bidirectional fast fills are implemented.

Example economics:

| Step                              | destination LiquidityVault balance |
|-----------------------------------|------------------------------------|
| <b>LP deposits</b>                | 100,000 USDC                       |
| <b>Fast fill to recipient</b>     | -999 USDC                          |
| <b>Balance while CCTP pending</b> | 99,001 USDC                        |
| <b>Canonical CCTP arrives</b>     | +1,000 USDC                        |
| <b>Post-settlement balance</b>    | 100,001 USDC                       |

The 1 USDC difference is gross execution revenue. It can be split between LP, solver and protocol. For the hackathon, keep accounting explicit but simple.

LP risks are source reorg/finality risk, CCTP or destination outage, contract risk, agent mispricing, one-way inventory drain and capital duration. Initiating CCTP before fast fill reduces the key risk from 'will settlement be started?' to 'will an already committed canonical settlement complete?'.

## 11. Settlement Agent and CCTP completion

- Monitor fast-filled but not canonically settled intents.
- Track the CCTP message/attestation lifecycle and destination receive.
- Submit any required destination transaction idempotently.
- On receive, correlate canonical funds to intentId/cctpRef.
- If FAST\_FILLED: credit/replenish LP inventory and realise fee accounting.
- If not FAST\_FILLED: route canonical amount to recipient.
- Emit settlement event and update final state.
- Never let database state substitute for onchain settlement state.

V1 should settle per intent for clarity and auditability. Netting and batching are production optimisations, not required for the hackathon path.

## 12. The Graph integration

The Graph must be visibly load-bearing in the qualifying path. Agents should consume live Graph-backed data for intent discovery and risk state. Removing the Graph should break automated discovery/decisioning, even though contracts remain authoritative for funds.

- Index IntentCreated and source CCTP correlation data.
- Index Arc FastFilled, settlement receive and LP vault events.
- Expose pending intents, available/reserved liquidity, unsettled exposure, settlement age and historical fills.
- Persist an agent decision record showing the exact Graph-derived inputs used for a quote.
- Do not use a static/mock feed for the submitted qualifying path.

## 13. Security model and explicit V1 trust assumption

V1 uses an authorised solver model. The destination LiquidityVault trusts signatures from allowlisted Circle Agent Wallet addresses after the agent has independently verified Ethereum. This is a deliberate hackathon trust assumption, not a claim of trustless crosschain verification.

| Threat                 | V1 mitigation                                                                                |
|------------------------|----------------------------------------------------------------------------------------------|
| Replay of fill         | Unique intentId + agent nonce + onchain consumed mapping.                                    |
| Compromised Graph data | Graph used for discovery; source transaction rechecked through Ethereum RPC.                 |
| Compromised agent      | Circle wallet policy + destination LiquidityVault caps + pause + per-intent/exposure limits. |
| Duplicate workers      | Idempotent state transitions and onchain replay protection.                                  |
| No solver available    | Canonical CCTP fallback pays recipient.                                                      |
| CCTP delayed           | Agent exposure cap / settlement-age policy pauses or reprices new fills.                     |
| Source reorg           | Confirmation threshold before fast fill.                                                     |
| Vault drain bug        | USDC-only allowlist, minimal contract surface, tests, caps                                   |

Post-hackathon trust minimisation may use a canonical crosschain verification primitive, quorum attestations or bonded/slashable solvers. Do not implement these before V1 works.

## 14. Business model and product position

Arcaidia monetises the time value and risk of destination liquidity, not the existence of canonical bridging. The user pays a fast-execution fee; LPs earn for advancing capital; the solver can receive a share for pricing/execution; the protocol can retain a share.

- Customer promise: express an outcome, receive destination USDC quickly, and let infrastructure settle underneath.
- LP value: fee income on short-duration inventory advances backed by already-initiated canonical settlement.
- Protocol moat: solver/risk data, liquidity network, execution policies and future multi-asset routing.
- V2 expands from 'USDC here → USDC there' to 'asset A here → desired asset B there' using Uniswap.
- V3 makes solver/discovery services machine-payable through Hedera/x402.

## 15. MVP definition of done

- Public GitHub repository with frequent, chronological commits.
- Ethereum testnet ArcaidiaIntentRouter successfully commits test USDC to CCTP and emits indexed intent.
- Live The Graph data reaches the Liquidity Agent.
- Agent independently verifies the Ethereum source transaction.
- Agent produces deterministic risk decision and signed FillAuthorization using a Circle Agent Wallet.
- ArcaidiaLiquidityVault fast-fills recipient and prevents replay.
- UI clearly distinguishes FAST\_FILLED from canonical SETTLED.
- CCTP destination completion replenishes the LP vault, or canonical fallback pays recipient if no fast fill occurred.
- At least one end-to-end automated test or scripted demo reproduces the lifecycle.
- README contains architecture, trust assumption, setup, contract addresses, data source, demo steps and targeted bounty mapping.

## 16. Concrete implementation architecture and software stack

This section is additive to the protocol design above. It removes implementation ambiguity for Claude Code: each architectural box is mapped to a physical software component, language, runtime, interface boundary and responsibility. V1 MUST be direction-agnostic. Ethereum and Arc are symmetric execution environments; either chain may be source or destination.

### 16.1 Repository and shared domain layer

Physical form: a pnpm TypeScript monorepo package imported by the frontend, agents, deployment scripts and integration tests. It contains no chain-specific business logic.

- Software: TypeScript, Node.js, pnpm.
- Owns: Intent, FillAuthorization, AgentDecision, RiskPolicy, VaultState, SettlementReference, SettlementStatus, ABIs, chain configuration, token configuration and deployed addresses.

- Rule: transfer direction is data/configuration, never a separate ETH-to-Arc or Arc-to-ETH implementation.

```
packages/domain/
  src/types.ts
  src/config.ts
  src/abis.ts
  src/intent-id.ts
  src/eip712.ts
```

## 16.2 Symmetric chain execution layer

Physical form: the same Solidity contracts deployed on both Ethereum and Arc. There MUST NOT be separate EthereumIntentRouter/ArcIntentRouter or EthereumVault/ArcVault implementations. The same ABI and logic execute on both chains.

- Software: Solidity, Foundry, OpenZeppelin contracts.
- Contracts: ArcaidiaIntentRouter.sol, ArcaidiaLiquidityVault.sol, SettlementReceiver.sol, MockUSDC.sol (development only).
- Ethereum deployment: IntentRouter + LiquidityVault + SettlementReceiver.
- Arc deployment: IntentRouter + LiquidityVault + SettlementReceiver.
- When Ethereum is source, Arc LiquidityVault fast-fills. When Arc is source, Ethereum LiquidityVault fast-fills.

```
ETH -> ARC: Ethereum IntentRouter -> solver -> Arc LiquidityVault -> CCTP reimburses Arc vault
ARC -> ETH: Arc IntentRouter -> solver -> Ethereum LiquidityVault -> CCTP reimburses Ethereum vault
```

## 16.3 Deterministic CREATE2 deployment

All symmetric Arcaidia protocol endpoints MUST be deployed through CREATE2 so corresponding contracts resolve to the same addresses on Ethereum and Arc wherever the underlying chains support the same CREATE2 semantics. Claude Code must treat same-address deployment as a V1 acceptance criterion, not an optional optimization.

- Use identical creation bytecode and deterministic salts for corresponding deployments.
- Chain-specific USDC addresses, CCTP domains and Circle contract addresses MUST NOT alter the CREATE2 init-code hash if identical endpoint addresses are required.
- Therefore use deterministic deployment followed by tightly controlled chain-specific initialization/configuration, or an equivalent pattern that preserves identical init code.
- Deployment tests must calculate expected CREATE2 addresses before broadcasting and assert the deployed address on both chains.

## 16.4 Asset configuration: MockUSDC and real USDC

Core protocol logic operates against one configured IERC20 settlement asset. MockUSDC and real USDC are not separate code paths. Development uses a mintable OpenZeppelin ERC20; production/qualifying configuration points the same protocol interfaces at real USDC.

```
local/dev config: settlementAsset = MockUSDC
production config: settlementAsset = USDC
```

Do not implement a runtime boolean such as useRealUSDC. Asset selection belongs to deployment/configuration so changing asset does not re-orchestrate the application or change agent logic. Both chain vaults are funded in development by freely minted MockUSDC, allowing bidirectional liquidity tests without appreciated assets.

## 16.5 Observation layer

Physical form: subgraphs/indexing definitions plus a TypeScript GraphQL client. The Graph is the load-bearing discovery/state plane for the qualifying V1 path, indexing both Ethereum and Arc.

- Software: The Graph, GraphQL, TypeScript.
- Indexes: IntentCreated, CCTP correlation/reference data available onchain, vault deposits/withdrawals, FillAuthorized/IntentFilled, settlement state, vault liquidity and outstanding exposure.
- The solver discovers work and obtains consolidated live state through GraphObservationProvider.
- Security boundary: The Graph is observation, not authorization. Before risking LP funds the solver independently verifies the source receipt/logs against an RPC.

```
ObservationProvider
| - InMemoryObservationProvider    // deterministic tests
|- GraphObservationProvider        // qualifying V1
```

## 16.6 Solver / Liquidity Agent

Physical form: an offchain Node.js/TypeScript service. It is the agent brain. It consumes live Graph state, independently verifies source-chain evidence through viem/RPC, applies deterministic risk and pricing rules, and returns ACCEPT/REJECT plus a fee/output decision.

- Software: Node.js, TypeScript, viem.
- Inputs: intent parameters, destination vault liquidity, outstanding unsettled exposure, utilization, CCTP health/backlog, user max fee and policy limits.
- Outputs: AgentDecision and, on acceptance, a typed FillAuthorization to be signed by the Agent Wallet.
- An LLM may explain or summarize decisions, but capital-safety decisions MUST remain deterministic and testable.

## 16.7 Circle Agent Wallet

Physical form: machine signing authority used by the offchain solver. The agent brain remains an ordinary service; Circle Agent Wallet supplies its bounded blockchain identity/authority. Its address is recognized by Arcadia contracts and its signed/initiated actions manifest onchain.

- Software: Circle Agent Stack / Agent Wallet SDK plus the TypeScript integration wrapper.
- Role: sign EIP-712 FillAuthorization and/or execute permitted transactions according to the supported Agent Wallet model.
- Policy: chain/contract allowlists, transaction limits and bounded spending/session controls where supported.

```
Solver Agent (offchain) -> Circle Agent Wallet (signing authority) -> destination LiquidityVault (onchain)
```

## 16.8 Settlement Agent and CCTP adapter

Physical form: an offchain Node.js worker plus a Circle CCTP adapter. It watches canonical transfers, obtains/handles the required Circle attestation/message lifecycle, completes destination settlement and reconciles whether canonical funds reimburse the destination LP vault or pay the recipient through the no-fast-fill fallback.

- Software: Node.js, TypeScript, viem, Circle CCTP APIs/contracts.
- The core domain depends on SettlementAdapter, not Circle-specific calls scattered through the codebase.

```
SettlementAdapter
| - MockSettlementAdapter          // unit/local E2E
|- CircleCCTPAdapter              // qualifying V1
```

## 16.9 User application

Physical form: browser application. Privy is integrated early, not bolted on after protocol completion.

- Software: Next.js, React, TypeScript, Privy, viem.

- Responsibilities: authentication/wallet, source and destination selection, balance, amount, recipient, max fee, intent submission, agent decision/status, fast-fill status and canonical settlement status.
- The UI must represent both directions with the same components and must never collapse FAST\_FILLED and SETTLED into one misleading completed state.

## 16.10 Test and CI layer

Foundry is the Solidity test/deployment tool. Vitest is used for TypeScript domain, agent and adapter tests. A deterministic local E2E harness proves the full economic lifecycle before sponsor integrations are substituted.

- Do not introduce Hardhat unless a sponsor SDK creates an unavoidable requirement.
- Every critical bidirectional scenario must be tested in both directions.

## 17. Component interdependencies and adapter boundaries

Claude Code MUST preserve the following dependency direction. External sponsor services sit behind interfaces and may be substituted without rewriting the core protocol. No unfinished downstream integration may block development of an upstream module.

```

Shared Domain / Config
|
+--> IntentRouter (same Solidity on ETH + Arc)
+--> LiquidityVault (same Solidity on ETH + Arc)
+--> Risk / Decision Engine
    |
    +--> ObservationProvider -> InMemory | The Graph
    +--> AgentSigner         -> Local   | Circle Agent Wallet
    +--> SettlementAdapter   -> Mock    | Circle CCTP

```

```

IntentRouter + LiquidityVault + Agent -> local golden E2E
local adapters are then replaced one at a time; core interfaces do not change.

```

Direction resolution MUST come from chain configuration: sourceChainId determines source router/RPC/CCTP domain; destinationChainId determines destination vault/RPC/CCTP domain. The solver implementation remains processIntent(intent), not processEthToArc() and processArcToEth().

## 18. CCTP availability as an agentic risk signal

Canonical settlement is intentionally asynchronous. Circle attestations can remain pending while source confirmations are reached, and delayed/stuck settlement must be treated as an economic exposure condition rather than hidden as an infrastructure inconvenience. The user fast path does not wait for canonical completion, but LP inventory remains exposed until CCTP reimbursement completes.

The Liquidity Agent SHOULD continuously derive or observe at minimum:

- oldestUnsettledIntent - age of the oldest fast-filled but canonically unsettled intent.
- pendingCCTPValue - aggregate principal currently advanced by LPs and awaiting canonical reimbursement.
- averageSettlementLatency - recent observed canonical settlement time, preferably with a rolling window.
- vaultLiquidity - currently deployable destination liquidity after reserve requirements.
- CCTP/Iris health - whether required Circle message/attestation services are reachable and progressing.

The risk engine then responds deterministically:

```

Circle normal      -> ACCEPT normally
Circle slowing     -> increase required fee and/or reduce maximum fill size
Backlog too large  -> REJECT new fills that would breach exposure policy
Circle unavailable  -> PAUSE new fast fills; Settlement Agent continues recovery/reconciliation

```

This behaviour is part of Arcaidia's agentic value proposition: the solver is not merely forwarding transactions. It actively prices and constrains the time/liquidity risk created by asynchronous canonical settlement. Thresholds must live in RiskPolicy and be covered by unit tests.

## 19. TDD development milestones and integration gates

Claude Code MUST execute V1 as gated work packages. A milestone is complete only when its tests and stated acceptance gate pass. Sponsor integrations are introduced only after the deterministic local lifecycle is green, with Privy deliberately introduced early so the human flow evolves alongside the protocol.

### M0 - Domain and repository contract

Stack: TypeScript + pnpm

Work: Create shared types/config, canonical intent encoding, status model, EIP-712 schema, errors and package boundaries.

Gate: All downstream packages consume the same domain definitions; no duplicated intent/status schemas.

### M1 - Symmetric chain core

Stack: Solidity + Foundry + OpenZeppelin

Work: Build MockUSDC, ArcaidiaIntentRouter, ArcaidiaLiquidityVault and SettlementReceiver once; deploy/test the same implementations as Ethereum and Arc instances.

Gate: Contract unit tests green; direction is configuration; CREATE2 expected addresses are deterministic.

### M2 - Bidirectional vault safety

Stack: Foundry

Work: Exercise deposits, withdrawals, reserve floors, signer authorization, fill caps, fees, pause, replay, insufficient liquidity and recipient transfer for both destination directions.

Gate: No LP principal can leave outside defined policy; ETH->Arc and Arc->ETH test matrices pass.

### M3 - Privy thin user flow

Stack: Next.js + React + TypeScript + Privy + viem

Work: Login/wallet, balances, source/destination selector, amount, recipient, max fee and intent submission against the working router.

Gate: A real Privy wallet can create an intent in either direction through the UI.

### M4 - Deterministic agent intelligence

Stack: Node.js + TypeScript + Vitest

Work: Implement pure evaluateIntent(intent,vaultState,settlementState,riskPolicy) including liquidity, exposure, fee and CCTP-health decisions.

Gate: All accept/reject/reprice/pause branches are deterministic and unit-tested.

### M5 - Fill authorization path

Stack: TypeScript + viem + Foundry

Work: Use LocalAgentSigner to produce EIP-712 FillAuthorization and have destination LiquidityVault verify signer, intent, nonce, expiry, amount, fee and policy.

Gate: Complete local fast-fill works in both directions; tamper/replay tests fail safely.

## **M6 - Mock canonical settlement**

Stack: Node.js + TypeScript + MockSettlementAdapter

Work: Implement Settlement Agent, canonical lifecycle, LP reimbursement and no-solver fallback without external Circle dependency.

Gate: Fast path and fallback path settle correctly in both directions; settlement is idempotent.

## **M7 - Golden local E2E**

Stack: Foundry + Vitest integration harness

Work: One command executes intent -> observe -> decide -> sign -> fast-fill -> canonical settlement -> reimbursement using mocks/local adapters.

Gate: The complete economic lifecycle passes deterministically before external sponsor integrations are required.

## **M8 - The Graph integration**

Stack: The Graph + GraphQL + TypeScript

Work: Replace InMemoryObservationProvider with GraphObservationProvider indexing live Ethereum and Arc contract state. Keep independent RPC verification before LP risk.

Gate: Disabling Graph stops automated discovery/processing; live Graph data materially changes agent decisions.

## **M9 - Circle Agent Wallet**

Stack: Circle Agent Stack + TypeScript

Work: Replace LocalAgentSigner with CircleAgentWalletSigner and apply supported machine-wallet policies.

Gate: Actual Agent Wallet authority signs/executes a bounded destination-chain fill without changing core agent logic.

## **M10 - Real CCTP and USDC configuration**

Stack: Circle CCTP + viem + deployment config

Work: Replace MockSettlementAdapter with CircleCCTPAdapter and point production configuration at real USDC. Preserve MockUSDC for deterministic tests.

Gate: Real canonical transfer can reimburse the opposite-chain LP vault; both directions validated on target networks.

## **M11 - Full sponsor E2E**

Stack: Privy + Graph + Circle Agent Stack + CCTP + Arc/Ethereum

Work: Run the qualifying path from Privy user intent through live Graph observation, solver decision, Agent Wallet fill and canonical reimbursement.

Gate: One repeatable end-to-end demo proves all V1 sponsor integrations are load-bearing.

## **M12 - Submission hardening**

Stack: Next.js + docs + CI

Work: Polish status/decision UI, transaction links, failure handling, architecture diagrams, README, contract addresses, setup, demo script and video.

Gate: Arc/Circle, The Graph and Privy submission acceptance checklist is fully evidenced.

## M13 - Freeze V1

Stack: Git/CI

Work: Tag/freeze the working V1 path. V2/V3 changes must not destabilize it.

Gate: All V1 tests and qualifying demo remain green before extension work starts.

## 20. V1 sponsor acceptance matrix and extension discipline

V1 is deliberately engineered so sponsor requirements are acceptance criteria rather than presentation claims. Before submission, the repository/demo must visibly prove the following.

### 20.1 Arc / Circle

- Arc is a real execution/settlement environment, not a logo or read-only integration.
- USDC is used meaningfully in the qualifying path; MockUSDC remains only for development/testing.
- Circle Agent Wallet is the machine authority for autonomous solver action.
- The agent has substantive decision logic tied to live liquidity, exposure, fee and CCTP-health signals.
- CCTP provides canonical settlement while Arcaidia provides the fast economic settlement layer.
- Functional frontend/backend, architecture diagram, public repository, documentation and demo are maintained as submission deliverables.

### 20.2 The Graph

- Live Graph data from both chains is load-bearing for automated intent discovery and state aggregation.
- The agent reasons/acts on Graph-derived state; static or mocked Graph data is not used in the qualifying demo.
- Graph is not treated as the LP security oracle; source evidence is independently verified before a fast fill.
- README/demo explicitly identifies the indexed entities/events and the agent decisions they influence.

### 20.3 Privy

- Privy is the user authentication/wallet layer for the actual financial flow.
- A Privy wallet creates a real Arcaidia intent and follows it through fast and canonical settlement states.
- The same UI supports Ethereum->Arc and Arc->Ethereum without separate flows.

### 20.4 V2 - Uniswap extension

V2 begins only after V1 is frozen. Add desiredToken and minimumOutput to the generalized intent and introduce an ExecutionAdapter with a Uniswap implementation. The solver chooses the route; the user specifies the desired outcome rather than swap calldata. Prefer keeping LP inventory centered on USDC and performing destination-side asset transformation where appropriate. V2 MUST reuse V1 routers/vaults/agent interfaces rather than fork the protocol.

### 20.5 V3 - Hedera/x402 extension

V3 adds a machine-commerce surface around the existing solver rather than making Hedera a new settlement dependency. Expose quote/route/execution services through an x402-gated service paid on Hedera, allowing

external agents to purchase Arcaidia solver capabilities. Implement behind a SolverCommerceAdapter so V1 continues operating unchanged if Hedera/x402 is absent.

## 21. Legacy concise build order (read with Section 19 milestones)

13. Scaffold monorepo, environment validation, public README and .env.example.
14. Implement Ethereum intent/CCTP router and contract tests.
15. Implement ArcaidiaLiquidityVault, replay protection, caps, pause and tests.
16. Stabilise contract events and intent/cctp correlation fields.
17. Build The Graph pipeline against live provider-backed data.
18. Build deterministic agent verifier/risk engine and decision JSON logs.
19. Integrate Circle Agent Wallet signing and EIP-712 FillAuthorization.
20. Build Settlement Agent and canonical receive/reimbursement path.
21. Build minimal Privy frontend and execution timeline.
22. Run full end-to-end path repeatedly; fix reliability before V2.
23. Only after V1 passes: add V2 Uniswap crosschain swap intents.
24. Do not start V3 Hedera/x402 unless V1 and V2 are stable.

## 22. Testing and invariants

- No fast fill without a unique source intent and verified CCTP commitment.
- No intent can be fast-filled twice.
- No expired FillAuthorization can execute.
- No unauthorised solver signer can move LP funds.
- No fill may breach per-intent, reserve-floor or total-unsettled-exposure limits.
- FAST\_FILLED and SETTLED remain distinct and independently observable.
- No-solver path results in canonical recipient delivery, not trapped funds.
- Settlement worker is idempotent across retries/restarts.
- Graph/database downtime can halt automation but cannot grant authority over funds.
- UI never labels canonical settlement complete before onchain confirmation.

## 23. ETHOnline submission discipline

Treat submission quality as part of the build.

- Start the ETHOnline submission early and keep project details current until close.
- Keep the project in a public GitHub repository.
- Commit and push frequently; avoid a single large final commit because judges manually review code.
- Record a 2-4 minute demo video at 720p or higher with clear spoken audio that is not music.
- Demo the actual onchain/offchain boundary: source CCTP commitment, Graph observation, agent decision, Circle Agent Wallet fill, recipient receipt, and later LP reimbursement.
- Show the agent's live decision inputs and explain the authorised-solver V1 trust assumption.
- Include architecture diagram, setup instructions, .env.example and a concise bounty mapping in README.

## 24. V2 and V3 extension contracts

V2 - Uniswap: generalise the intent outcome to desiredToken + minimumOutput. The user specifies WHAT they want on the destination; the solver chooses HOW to fulfil it. Destination LP liquidity can remain USDC and be transformed through Uniswap before delivery. Do not encode Uniswap-specific calldata in the user's source intent.

V3 - Hedera/x402: expose solver discovery/quote/execution as machine-payable services. An autonomous agent receives HTTP 402, pays via the Hedera/x402 path, receives a route/quote and can then invoke Arcaidia. This should remain isolated from V1 contracts.

## 25. Final V1 judge pitch

**Arcaidia does not replace canonical settlement.** The user's USDC is committed to Circle CCTP first. Autonomous liquidity agents observe that committed settlement, independently verify it, price the risk, and decide whether to advance destination liquidity from an destination LiquidityVault. The recipient receives USDC quickly; CCTP completes asynchronously and reimburses the LP. Onchain contracts hold truth and funds, The Graph supplies live observation, and Circle Agent Wallets provide bounded autonomous execution authority.

That is the V1 architecture. Do not broaden scope until this path is demonstrably reliable.